

Hidden Frame-Pack Animation & Inspect Deterrent

A reusable AI prompt to add both features to any web project. Paste the grey block into your coding assistant (Claude Code, Cursor, etc.).

How to use this

- Put your animation frames in one folder, sequentially numbered — e.g. `frame_0001.jpg`, `frame_0002.jpg`, ... A short capture at 30 fps (150–300 frames) works best.
- In the prompt below, replace `<<<FRAMES_FOLDER>>>` with that folder's path.
- Paste the whole grey block into your AI assistant inside the target project.
- After it builds, run the generated pack script (e.g. `npm run pack:frames`) whenever the frames change.

What the visitor sees. The animation loads as a *single* request for one binary file (`s-<hash>.bin`) whose bytes are not a valid image or video container. The Network tab shows one `fetch`, not images, not a sprite sheet, not a video, not a 3D model. The frames are never addressable as URLs, and the DevTools response preview is noise. On screen it renders into a `<canvas>`, so the Elements panel shows only an empty canvas element.

Be realistic. This is a strong deterrent against casual copying and scraping, not encryption. The unmask key ships in the JavaScript bundle, so a determined person can still recover the frames. It stops the one-click / one-glance cases: right-click → save, "it's obviously just images", renaming the file to `.webp`, and drag-out.

The prompt [COPY EVERYTHING BELOW](#)

Implement TWO production-quality features in this project. Read any framework docs in the repo before writing code. If this is a Next.js App Router project, follow its conventions; otherwise adapt the same architecture to the stack in use.

FEATURE 1 – Scroll-scrubbed frame animation, shipped as ONE opaque binary

WHAT I WANT

A sequence of still frames (a looping character / product / robot animation) that is scrubbed by the user's scroll position and painted into a `<canvas>`. Critically:

- The browser must make EXACTLY ONE network request for the whole animation.
- That request must be a single binary file (e.g. `s-<hash>.bin`) whose bytes are NOT a recognizable image / video container – it must not be obvious from the Network tab that this is images, a sprite sheet, a video, or a 3D model.
- Individual frames must never be addressable as URLs.
- It must still stream progressively (playback starts before full download).
- Serve the binary with a generic `'Content-Type: application/octet-stream'` and an immutable long-lived cache header. No filename or header should hint at "frames", "sprites", "video", etc.

SOURCE FRAMES

Located at: `<<<FRAMES_FOLDER>>>`

Sequentially numbered images. Use a numeric-aware sort (frame_9 before frame_10).

BUILD SCRIPT (e.g. scripts/pack-frames.mjs, exposed as an npm script "pack:frames")

Use `sharp`. The script must:

1. List source frames in true order (numeric collator). Optionally cut to a LAST_FRAME and/or sample every Nth (FRAME_STRIDE) – expose both as constants.
2. Re-encode each frame to WebP at a target width (default 1920, quality ~74, `withoutEnlargement`). Encode ONE frame at a time (memory).
3. Read width / height from the first encoded frame.
4. Build an unmasked container with this EXACT layout:

offset 0	u32 LE	frame count
offset 4	u16 LE	frame width
offset 6	u16 LE	frame height
offset 8	u32 LE	× frameCount

byte length of each frame payload
then the WebP payloads, back-to-back, in play order
5. Generate a fresh random 32-byte XOR key every build and XOR-mask the whole container in place (XOR is its own inverse).
6. Name the output `s-<sha256(maskedBytes) sliced to 16 hex>.bin`, delete any previous pack, write it to the public/static dir (e.g. public/_seq/).
7. Emit a manifest JSON (committed, imported by a server component):

```
{ url, frameCount, width, height, bytes, poster }
```

where `poster` is a tiny (~640px, q60) WebP of frame 0 as a `data:` URI for instant first paint.
8. Emit the mask key as a SEPARATE generated JS module (e.g. `export const MASK\_KEY = "<base64>";`) that the CLIENT imports directly. Do NOT put the key in the manifest – manifest fields become props and get serialized into the page HTML in the clear; the key must land in a JS chunk.

CLIENT LOADER (e.g. framePack.js)

A `loadFramePack(manifest, { onFrame, signal })` function that:

- `fetch`es manifest.url with the abort signal.
- Reads Content-Length to preallocate one Uint8Array; if missing, buffer the whole body then process (correctness over progressiveness).
- Streams via `response.body.getReader()`. As each chunk lands: copy into the buffer, XOR-unmask only the newly filled byte range (key stays in phase via `i % key.length` over the absolute offset).
- Once the fixed header + length table has arrived, parse the per-frame byte ranges ONCE.
- Whenever a frame's full payload has arrived, decode it and emit it via `onFrame(index, frame)` strictly in play order. Decode with `createImageBitmap(new Blob([bytes.slice()], { type: "image/webp" }));` fall back to an + object URL for browsers without createImageBitmap. Await decodes one at a time so a fast connection can't queue hundreds of concurrent decodes.

PLAYER COMPONENT (client component, canvas-based)

- Outer <section> height = `calc(100dvh + var(--seq-scroll-screens) \* 100dvh)` (a sticky screen + a scroll runway measured in viewport-heights, not pixels). Expose `--seq-scroll-screens` as a CSS var (e.g. 2 on desktop, ~1.25 on mobile).
- Inside: a `position: sticky; top: 0; height: 100dvh` stage containing the <canvas> (absolutely positioned, fills the stage) and, over it, page copy.
- Scroll progress = clamp(-section.getBoundingClientRect().top / (section.offsetHeight - window.innerHeight), 0, 1).
- Target frame = round(progress * (frameCount - 1)).
- A requestAnimationFrame loop that WALKS the playhead toward the target: snap when within ~0.4 of a frame; otherwise step by clamp(distance*EASE, MIN_STEP, MAX_STEP) with MIN_STEP = 1, MAX_STEP ≈ 6 – this guarantees no frame is skipped on a coarse mouse wheel while never lagging unboundedly on a fling or jump-to-anchor.
- Paint to canvas with a plain STATIC "cover" fit: `scale = max(cw/iw, ch/ih)`, position depends ONLY on stage size (no zoom, pan, or per-frame adjustment) so the subject holds exactly the same size / position for the whole scroll – only the pixels change. DPR-aware backing store (cap DPR at 2), ResizeObserver to re-fit on resize / rotate, `imageSmoothingQuality = "high"`.

- The `<canvas>` must be `'aria-hidden="true"'` with no alt text or class name that describes the content (no "robot", "frames", etc.).
- If a frame isn't decoded yet, draw the nearest decoded frame (search outward), falling back to the poster.
- Run the rAF loop only while the section intersects the viewport (`IntersectionObserver`).
- Show the inline `'poster'` data-URI (a plain `<img>`) until the canvas has painted its first real frame, then hide it.
- `'prefers-reduced-motion: reduce'` → collapse the section to `'100dvh'`, no runway, just hold frame 0.
- Cleanup: abort the fetch and call `'.close()'` on every `ImageBitmap` on unmount.

RATIONALE (so you make the right calls)

- A sprite sheet is impossible: iOS Safari caps a single decoded image at ~16.7 Mpx, so any sheet large enough to look good is blank on iPhone.
- A `<video>` scrubbed by `currentTime` stutters badly under scroll on iOS Safari (async seeks, coalesced decodes).
- Packing still frames keeps exact frame-accurate scrubbing, decodes off the main thread, and shows as exactly one request.
- The XOR mask is a speed bump against casual copying, not encryption (the key ships in the bundle). It stops the file being renamed to `.webp` and opened, and makes the DevTools response preview noise.

COMMIT the manifest JSON and the packed `.bin`. Keep source frames reasonably sized (convert huge PNGs to ~quality-90 JPEG first – the packer re-encodes to WebP anyway).

FEATURE 2 – Inspect / DevTools deterrent

A small client component, mounted ONCE high in the tree (root layout). In a `useEffect` it adds window listeners (cleaned up on unmount):

- `'contextmenu'` → `preventDefault` (blocks right-click "Save image as" / "Inspect")
- `'keydown'` (capture phase) → `preventDefault` when the event is:
 - F12
 - `Ctrl/Cmd + Shift + I / J / C` (inspector, console, element picker)
 - `Ctrl/Cmd + U` (view source)
- `'dragstart'` → `preventDefault` when `'event.target instanceof HTMLImageElement'` (blocks drag-out of `<img>` assets)

Leave text selection and copy working – right-click is also how people reach accessibility tooling, so don't break more than the context menu.

Add a header comment stating plainly what this does NOT and CANNOT stop: opening DevTools from the browser menu, `'view-source:'`, curl, disabling JavaScript, or reading the decoded canvas. It is a deterrent for casual users only.

OPTIONAL: gate it behind `'process.env.NODE_ENV === "production"'` so it is inactive during local development.

DELIVERABLES

- the build script + npm script
 - the generated manifest JSON & mask-key module
 - the client loader
 - the player component
 - the deterrent component
 - both wired into the app
 - a short note on how to regenerate the pack when frames change
-

